

Library Book Search Engine

A Comparative Study of Binary Search Trees and Hash Tables

This project explores the design and implementation of a highly scalable search engine for library databases. By engineering a dual-architecture indexing system utilizing both Binary Search Trees and Hash Tables, the system achieves an optimal balance between ordered data retrieval and constant-time search efficiency.

Mann Chauhan

B.Tech Mathematics and Computing (MnC) - Semester 2
Dhirubhai Ambani Institute of Information and Communication
Technology (DA-IICT)

1 Problem Statement

Modern library databases manage large amounts of records. So, searching for a large amounts of records should be fast. A naive search (scanning each record) is too slow for big datasets. Therefore we need to optimize search techniques so that queries by title or author do not require scanning the entire dataset for this we use efficient data structures. A BST provides $\mathcal{O}(\log n)$ search time, while a hash table offers average $\Theta(1)$ search time. By using both, we get sorted order (from the BST) and constant-time lookup (from hashing). So, by this we get efficient indices (a BST and a hash table) so that searches run quickly even for thousands of books. This combination avoids slow scans and makes searching very quick.

2 Design and Data Structures

2.1 Evolution from ID to Title/Author Search

Initially, the design considered searching for books using unique numerical IDs. However, from a user-experience perspective, library searches using book Titles and Authors (like our Collage's Library), not arbitrary numbers. Therefore, our design should be based to titles and authors the dataset based on alphanumeric strings.

2.2 The Book Struct and Multi-Pointer Architecture

To optimize search queries, the system stores books using two distinct data structures simultaneously. Each book is stored in a **Book struct** containing the fields: **title**, **author**, **year**, and **status** (Available or Currently Borrowed). To avoid allocating duplicate memory for the same book across different structures, the struct utilizes a multi-pointer architecture:

- **Title BST:** **tleft**, **tright** point to children. **tnxBST** chains duplicate titles together at the same node.
- **Author BST:** **aleft**, **aright** point to children. **anxBST** chains duplicate authors.
- **Hash Tables:** **tnxHash** links books in the same title bucket, and **anxHash** links books in the author bucket.

2.3 Dual Indexing System

To optimize search queries, the system stores books using two distinct data structures simultaneously. Since a user might search by Title, by Author, or a combination of both, we implemented:

- **Hash Tables (HT):** Two separate arrays of pointers (**THT** for Titles, **AHT** for Authors) to achieve $\mathcal{O}(1)$ average-case retrieval.
- **Binary Search Trees (BST):** Two separate trees (**troot** for Titles, **aroot** for Authors) to achieve $\mathcal{O}(\log N)$ average-case retrieval and maintain a sorted structure.

2.4 Data Normalization (For handling the user's search strings)

To ensure search accuracy, we separated the *stored data* from the *search logic*. Books are stored exactly as inputted. However, during tree traversal or hash generation, the strings are passed through a custom `asLower()` function. When displaying the results to the user, an `asTitle()` function dynamically capitalizes the first letter of each word to ensure clean UI presentation.

3 Implementation Challenges & Solutions

3.1 Hash Table Collision Management

We utilized a polynomial rolling hash function with a prime modulus of 13,681 to accommodate the large dataset efficient while keeping the collisions as possible as less amount. Collisions are handled via Separate Chainin; each `Book` struct contains `tnxHash` and `anxHash` pointers to act as nodes in a linked list.

3.2 BST: Iterative Insertion & Duplicate Handling

When importing real-world datasets that are often pre-sorted alphabetically, standard recursive BST insertion leads to a highly unbalanced tree (a straight line and becomes skew BST) resulting in a Stack Overflow.

Solution: We implemented an iterative `while(1)` loop for insertion, which uses zero stack memory and prevents crashes. Additionally, because a library may hold multiple books by the same author or multiple copies of the same title, exact matches in the BST are appended as a linked list to the matching node using the `tnxBST` and `anxBST` pointers. By using this we also prevents the BST to becomes skew.

3.3 Real-World Functionality

- **Borrow/Return (Soft Deleting):** Instead of permanently deleting nodes from the data structures when a book is checked out, we implemented a `status` boolean (1 for available, 0 for borrowed) because every book have 8 pointers so for deleting we manage this 8 pointers which is difficult. This acts as a "soft delete," maintaining the integrity of the pointers while reflecting real-world library operations. To reflect real-world library operations, books are not physically deleted from the data structures when checked out. Instead, the `status` boolean is toggled: it becomes `0` when a book is borrowed, and `1` when it is returned.
- **Safe Data Parsing:** To handle messy CSV data containing hidden carriage returns (`\r`) and stray symbols, we wrote a custom `safestoi()` function that mathematically extracts only valid digits, preventing fatal `std::invalid_argument` exceptions.
- **Memory Management:** To prevent memory leaks upon exit, a `Clean_Memory()` function systematically traverses the Hash Tables and deletes all dynamically allocated heap memory.

3.4 File Handling: CSV Import and Database Rewriting

A critical challenge was handling the massive 13,000+ book dataset. We implemented **Option 24** to import books from a raw CSV file (`books.csv`). This option only needs to be executed **once** to parse the data into the Hash Tables and BSTs. It then saves the entire clean database to a local text file (`library_data.txt`). On all subsequent runs, the CSV is no longer needed, and the program loads directly from the text file.

To ensure data persistence during the Borrow and Return processes, a `Rewrite_file()` function was implemented. Whenever a user changes a book's status (0 or 1), `Rewrite_file()` loops through the entire Hash Table and overwrites `library_data.txt` with the updated statuses, ensuring transactions are permanently saved.

4 Performance and Collision Analysis

The project includes a built-in analysis module (Option 8) to compare the efficacy of our hashing algorithm. By keeping the hash array size at the prime number 13,681, we achieved an optimal distribution of books across the indices, minimizing the maximum chain length and preserving near $O(1)$ lookup times.

(See the Outputs section for exact collision metrics).

Below is a table of time complexity comparisons (with n = number of books) comparing the theoretical speeds of our implemented data structures:

Operation	BST (Average)	BST (Worst)	Hash (Average)	Hash (Worst)
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\Theta(1)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\Theta(1)$	$\mathcal{O}(n)$

Table 1: Time Complexity Comparison

4.1 Load Factor and Time Complexity

The efficiency of our Hash Table is heavily dependent on its Load Factor (α). For our dataset of approximately 13,408 books, we initialized our Hash Table arrays with a prime number size of 13,681.

$$LoadFactor(\alpha) = \frac{TotalBooks(tbooks)}{TableSize} \approx \frac{13408}{13681} \approx 0.98$$

So, in our project both hashtable have 0.98 load factor. Therefore average time for successful search is $\mathcal{O}(1 + \alpha) = 1.98$

5 Learning Outcomes

1. **Algorithm Limitations when Constraints are big:** We learned the practical limitations of recursive algorithms when dealing with massive, real-world data limits, necessitating the shift to iterative BST insertion.
2. **Complex Structs:** We understand how utilizing a single struct with multiple pointers (left/right for BSTs, next for Hash Tables), allowing one allocated memory block to exist in four different places simultaneously.

3. **Tree Balancing Limitations:** We realized that while our iterative insertion prevents stack overflows, a standard BST does not self-balance. If the dataset is naturally sorted, the tree can degenerate into a skewed structure, degrading the worst-case search time to $\mathcal{O}(n)$. Upgrading to a self-balancing structure, such as an AVL Tree, would guarantee optimal $\mathcal{O}(\log n)$ performance across all scenarios.
4. **BSTs and Hash Tables:** Building pointer-based BSTs and hash chains reinforced our understanding of trees and handling collisions via separate chaining in Hash Table and for managing more than one copies of books in BST we use linked list.
5. **Data Persistence:** We learned how to manage database states by bridging RAM data structures with persistent text files using parsing techniques and the `Rewrite_file()` logic.
6. **The Mathematics of Hashing:** We learned that the choice of table size is not arbitrary. By selecting 13,681 (a prime number), we reduced common divisor in hash modulo arithmetic.
7. **Time Complexity:** By allowing our Load Factor ($\frac{tbooks}{13681}$) to approach 0.98 within a Separate Chaining architecture, we learned how to achieve constant $\Theta(1.98)$ search times without needlessly overallocating massive blocks of RAM.

Conclusion: Using a BST and Hash Table together makes this library search engine highly efficient and scalable. We try to make implementation fully meets the project goals, demonstrating the importance of memory management, data persistence, and choosing the right data structures.

6 Source Code

Below is the complete C++ implementation of the Library Book Search Engine.

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <algorithm>
5  #include <cctype>
6  using namespace std;
7
8  typedef long long ll;
9  double tbooks=0;
10
11 string asLower(string st) {
12     transform(st.begin(), st.end(), st.begin(), ::tolower);
13     return st;
14 }
15
16 string asTitle(string st) {
17     if (st.empty()) return st;
18     string tm = asLower(st);
19     tm[0] = toupper(tm[0]);
20     for (int i = 1; i < tm.size(); i++) {
21         if (tm[i - 1] == ' ') tm[i] = toupper(tm[i]);
22     }
23     return tm;
24 }
25
26 typedef struct Book {
27     string title;
28     string author;
29     int year;
30     bool status = 1;
31
32     Book *tleft = NULL;
33     Book *tright = NULL;
34     Book *tnxHash = NULL;
35     Book *tnxBST = NULL;
36
37     Book *aleft = NULL;
38     Book *aright = NULL;
39     Book *anxHash = NULL;
40     Book *anxBST = NULL;
41 } Book;
42
43 Book *THT[13681] = {NULL};
44 Book *AHT[13681] = {NULL};
45
46 int hash_fun(string st) {
47     st = asLower(st);
48     ll val = 0;
```

```

49     for (char ch : st) val = ((val * 31) + ch) % 13681;
50     return val;
51 }
52
53 void Print_Book(Book *b, int ct) {
54     cout << "\n----- Result # " << ct << " ----- \n";
55     cout << "Title   : " << asTitle(b->title) << '\n';
56     cout << "Author  : " << asTitle(b->author) << '\n';
57     cout << "Year    : " << b->year << '\n';
58     cout << "Status  : " << (b->status ? "Available\n" : "
    Currently Borrowed\n");
59     cout << "----- \n";
60 }
61
62 void in_Hash_by_title(Book *nb) {
63     int idx = hash_fun(nb->title);
64     if (THT[idx] == NULL) THT[idx] = nb;
65     else {
66         nb->tnxHash = THT[idx];
67         THT[idx] = nb;
68     }
69 }
70
71 Book *in_BST_by_title(Book *root, Book *nb) {
72     if (root == NULL) return nb;
73     Book *tm = root;
74     string nbTit = asLower(nb->title);
75     while (1) {
76         string rTit = asLower(tm->title);
77         if (nbTit < rTit) {
78             if (tm->tleft == NULL) { tm->tleft = nb; break; }
79             tm = tm->tleft;
80         } else if (nbTit > rTit) {
81             if (tm->tright == NULL) { tm->tright = nb; break; }
82             tm = tm->tright;
83         } else {
84             nb->tnxBST = tm->tnxBST;
85             tm->tnxBST = nb;
86             break;
87         }
88     }
89     return root;
90 }
91
92 void Search_Hash_Title(string tit) {
93     int idx = hash_fun(tit), ct = 0;
94     Book *ptrb = THT[idx];
95     tit = asLower(tit);
96     cout << "Searching for '" << asTitle(tit) << "' in Hash table
    ... \n";
97     while (ptrb != NULL) {

```

```

98         if (asLower(ptrb->title) == tit) {
99             ct++;
100             Print_Book(ptrb, ct);
101         }
102         ptrb = ptrb->tnxHash;
103     }
104     if (!ct) cout << "No books are found with title '" << asTitle
105         (tit) << "'\n";
106     else cout << "Total books found with '" << asTitle(tit) << "'
107         is : " << ct << '\n';
108 }
109
110 void Search_BST_Title(Book *root, string tit) {
111     if (root == NULL) {
112         cout << "No books are found with title '" << asTitle(tit)
113             << "'\n";
114         return;
115     }
116     string rTit = asLower(root->title);
117     if (tit < rTit) Search_BST_Title(root->tleft, tit);
118     else if (tit > rTit) Search_BST_Title(root->tright, tit);
119     else {
120         int ct = 1;
121         Print_Book(root, ct);
122         Book *ptrb = root->tnxBST;
123         while (ptrb != NULL) {
124             ct++;
125             Print_Book(ptrb, ct);
126             ptrb = ptrb->tnxBST;
127         }
128         cout << "Total books found with '" << asTitle(tit) << "'
129             is : " << ct << '\n';
130     }
131 }
132
133 void Analyze_Collisions(int type) {
134     int i, emp = 0, oneb = 0, moreb = 0, mxlen = 0;
135     for (i = 0; i < 13681; i++) {
136         Book *ptrb = (type == 1 ? THT[i] : AHT[i]);
137         if (ptrb == NULL) emp++;
138         else {
139             int len = 0;
140             while (ptrb != NULL) {
141                 len++;
142                 ptrb = (type == 1 ? ptrb->tnxHash : ptrb->anxHash
143                     );
144             }
145             if (len == 1) oneb++;
146             else moreb++;
147             mxlen = max(mxlen, len);
148         }
149     }

```



```

144     }
145     double loadfactor=(tbooks)/13681;
146     string tname = (type == 1 ? "Title" : "Author");
147     cout << "\n=== " << tname << "Hash Table Collision Analysis
148         ===\n";
149     cout << "Total Table Size          : " << 13681 << '\n';
150     cout << "Empty Slots                : " << emp << '\n';
151     cout << "Slots with 1 Book              : " << oneb << '\n';
152     cout << "Slots with collisions          : " << moreb << '\n';
153     cout << "Longest Chain Length          : " << mxlen << '\n';
154     cout << "Load Factor                  : " << loadfactor << '\n';
155     cout << "=====\n";
156 }
157 void in_Hash_by_Author(Book *nb) {
158     int idx = hash_fun(nb->author);
159     if (AHT[idx] == NULL) AHT[idx] = nb;
160     else {
161         nb->anxHash = AHT[idx];
162         AHT[idx] = nb;
163     }
164 }
165
166 Book *in_BST_by_Author(Book *root, Book *nb) {
167     if (root == NULL) return nb;
168     Book *tm = root;
169     string nbAut = asLower(nb->author);
170     while (1) {
171         string rAut = asLower(tm->author);
172         if (nbAut < rAut) {
173             if (tm->aleft == NULL) { tm->aleft = nb; break; }
174             tm = tm->aleft;
175         } else if (nbAut > rAut) {
176             if (tm->aright == NULL) { tm->aright = nb; break; }
177             tm = tm->aright;
178         } else {
179             nb->anxBST = tm->anxBST;
180             tm->anxBST = nb;
181             break;
182         }
183     }
184     return root;
185 }
186
187 void Search_Hash_Author(string aut) {
188     int idx = hash_fun(aut), ct = 0;
189     Book *ptrb = AHT[idx];
190     aut = asLower(aut);
191     cout << "Searching for '" << asTitle(aut) << "' books in Hash
192         table...\n";
193     while (ptrb != NULL) {

```

```

193         if (asLower(ptrb->author) == aut) {
194             ct++;
195             Print_Book(ptrb, ct);
196         }
197         ptrb = ptrb->anxHash;
198     }
199     if (!ct) cout << "NO books are found with author '" <<
200         asTitle(aut) << "'\n";
201     else cout << "Total books found with '" << asTitle(aut) << "'
202         is : " << ct << '\n';
203 }
204
205 void Search_BST_Author(Book *root, string aut) {
206     if (root == NULL) {
207         cout << "No books are found with author '" << asTitle(aut)
208             << "'\n";
209         return;
210     }
211     string rAut = asLower(root->author);
212     if ((aut < rAut)) Search_BST_Author(root->aleft, aut);
213     else if (aut > rAut) Search_BST_Author(root->aright, aut);
214     else {
215         int ct = 1;
216         Print_Book(root, ct);
217         Book *ptrb = root->anxBST;
218         while (ptrb != NULL) {
219             ct++;
220             Print_Book(ptrb, ct);
221             ptrb = ptrb->anxBST;
222         }
223         cout << "Total books found with '" << asTitle(aut) << "'
224             is " << ct << '\n';
225     }
226 }
227
228 void Search_Hash_Both(string tit, string aut) {
229     int idx = hash_fun(tit), ct = 0;
230     Book *ptr = THT[idx];
231     tit = asLower(tit), aut = asLower(aut);
232     cout << "Searching in Hash table for '" << asTitle(tit) << "'
233         ' and '" << asTitle(aut) << "'\n";
234     while (ptr != NULL) {
235         if (asLower(ptr->title) == tit && asLower(ptr->author) ==
236             aut) {
237             ct++;
238             Print_Book(ptr, ct);
239         }
240         ptr = ptr->tnxHash;
241     }
242     if (!ct) cout << "No book is found for this combination\n";
243     else cout << "Total books found with '" << asTitle(tit) << "'

```

```

238     wrriten by '" << asTitle(aut) << "' is : " << ct << '\n';
239 }
240 void Search_BST_Both(Book *root, string tit, string aut) {
241     if (root == NULL) {
242         cout << "No book is found for this combination\n";
243         return;
244     }
245     string rTit = asLower(root->title);
246     if (tit < rTit) Search_BST_Both(root->tleft, tit, aut);
247     else if (tit > rTit) Search_BST_Both(root->tright, tit, aut);
248     else {
249         int ct = 0;
250         if (asLower(root->author) == aut) {
251             ct++;
252             Print_Book(root, ct);
253         }
254         Book *ptr = root->tnxBST;
255         while (ptr != NULL) {
256             if (asLower(ptr->author) == aut) {
257                 ct++;
258                 Print_Book(ptr, ct);
259             }
260             ptr = ptr->tnxBST;
261         }
262         if (!ct) cout << "Found the title, but not this specific
                author\n";
263         else cout << "Total books found with '" << asTitle(tit)
                << "' wrriten by '" << asTitle(aut) << "' is : " << ct
                << '\n';
264     }
265 }
266
267 void Clean_Memory() {
268     for (int i = 0; i < 13681; i++) {
269         Book *ptr = THT[i];
270         while (ptr != NULL) {
271             Book *ptrn = ptr->tnxHash;
272             delete ptr;
273             ptr = ptrn;
274         }
275         THT[i] = NULL;
276     }
277 }
278
279 void saveBookToFile(Book *b) {
280     ofstream file("library_data.txt", ios::app);
281     if (file.is_open()) {
282         file << b->title << "|" << b->author << "|" << b->year <<
                "|" << b->status << '\n';
283         file.close();

```

```

284     }
285 }
286
287 int safestoi(string s) {
288     int res = 0;
289     for (char c : s) {
290         if (c >= '0' && c <= '9') res = (res * 10) + (c - '0');
291     }
292     return res;
293 }
294
295 void loadBooksFromFile(Book *tRoot, Book *aRoot) {
296     ifstream file("library_data.txt");
297     string tit, aut, year, stat;
298     if (file.is_open()) {
299         cout << "Loading books from database...\n";
300         while (getline(file, tit, '|') && getline(file, aut, '|')
301             && getline(file, year, '|') && getline(file, stat)) {
302             if (tit.empty()) continue;
303             Book *nb = new Book();
304             nb->title = tit;
305             nb->author = aut;
306             nb->year = safestoi(year);
307             nb->status = stat == "1" ? 1 : 0;
308             tbooks++;
309             in_Hash_by_title(nb);
310             in_Hash_by_Author(nb);
311             tRoot = in_BST_by_title(tRoot, nb);
312             aRoot = in_BST_by_Author(aRoot, nb);
313         }
314         file.close();
315         cout << "Database loaded successfully!\n";
316     } else cout << "No existing database found. A new one will be
317         created.\n";
318 }
319
320 void Rewrite_file() {
321     ofstream file("library_data.txt", ios::trunc);
322     if (file.is_open()) {
323         for (int i = 0; i < 13681; i++) {
324             Book *ptr = THT[i];
325             while (ptr != NULL) {
326                 file << ptr->title << "|" << ptr->author << "|"
327                     << ptr->year << "|" << ptr->status << '\n';
328                 ptr = ptr->tnxHash;
329             }
330         }
331         file.close();
332     }
333 }

```

```

332 void Borrow(string tit, string aut) {
333     int idx = hash_fun(tit);
334     Book *ptr = THT[idx];
335     while (ptr != NULL) {
336         if (asLower(ptr->title) == tit && asLower(ptr->author) ==
            aut) {
337             if (ptr->status) {
338                 ptr->status = 0;
339                 cout << "You borrowed '" << asTitle(tit) << "'
                    wrritten by '" << asTitle(aut) << "'\n";
340                 Rewrite_file();
341                 return;
342             } else {
343                 cout << "''" << asTitle(tit) << "' wrritten by '"
                    << asTitle(aut) << "' is currently borrowed by
                    someone else\n";
344                 break;
345             }
346         }
347         ptr = ptr->tnxHash;
348     }
349     cout << "Your book isn't found in our Library\n";
350 }
351
352 void Return(string tit, string aut) {
353     int idx = hash_fun(tit);
354     Book *ptr = THT[idx];
355     while (ptr != NULL) {
356         if (asLower(ptr->title) == tit && asLower(ptr->author) ==
            aut) {
357             if (!(ptr->status)) {
358                 ptr->status = 1;
359                 cout << "You returned '" << asTitle(tit) << "'
                    wrritten by '" << asTitle(aut) << "'\n";
360                 Rewrite_file();
361                 return;
362             } else {
363                 cout << "''" << asTitle(tit) << "' wrritten by '"
                    << asTitle(aut) << "' is already in the
                    library\n";
364                 return;
365             }
366         }
367         ptr = ptr->tnxHash;
368     }
369     cout << "This book is not belong to our library!\n";
370 }
371
372 void importFromCSV(Book *&tRoot, Book *&aRoot) {
373     ifstream file("books.csv");
374     string tit, aut, year, dummyLine;

```

```

375     int importedCount = 0;
376     if (file.is_open()) {
377         cout << "Importing perfectly clean dataset...\n";
378         getline(file, dummyLine);
379         while (getline(file, tit, ',') && getline(file, aut, ',')
380             && getline(file, year)) {
381             if (tit.empty()) continue;
382             Book *nb = new Book();
383             nb->title = tit;
384             nb->author = aut;
385             nb->year = safestoi(year);
386             nb->status = 1;
387             in_Hash_by_title(nb);
388             in_Hash_by_Author(nb);
389             tRoot = in_BST_by_title(tRoot, nb);
390             aRoot = in_BST_by_Author(aRoot, nb);
391             importedCount++;
392         }
393         file.close();
394         Rewrite_file();
395         cout << "Successfully imported " << importedCount << "
396             books!\n";
397     } else cout << "Could not find 'books.csv'.\n";
398 }
399
400 int main() {
401     Book *troot = NULL, *aroot = NULL;
402     loadBooksFromFile(troot, aroot);
403     int ch = 0;
404     string tit, aut;
405     while (ch != 11) {
406         cout << "\n===== \n";
407         cout << "LIBRARY BOOK SEARCH ENGINE\n";
408         cout << "===== \n";
409         cout << "1. Add new Book\n";
410         cout << "2. Search by title (HT)\n";
411         cout << "3. Search by title (BST)\n";
412         cout << "4. Search by author (HT)\n";
413         cout << "5. Search by author (BST)\n";
414         cout << "6. Search by title and author (HT)\n";
415         cout << "7. Search by title and author (BST)\n";
416         cout << "8. Analyze hash Collisions\n";
417         cout << "9. Borrow Book\n";
418         cout << "10. Return Book\n";
419         cout << "11. Exit\n";
420         cout << "24. Import new csv file books\n";
421         cout << "Enter your choice (1-11) : ";
422         cin >> ch;
423
424         switch (ch) {
425             case 1: {

```

```

424         Book *nb = new Book();
425         cout << "\nEnter Title : ";
426         getline(cin >> ws, nb->title);
427         cout << "Enter Author : ";
428         getline(cin >> ws, nb->author);
429         cout << "Enter Year : ";
430         cin >> nb->year;
431         tbooks++;
432         in_Hash_by_title(nb);
433         in_Hash_by_Author(nb);
434         troot = in_BST_by_title(troot, nb);
435         aroot = in_BST_by_Author(aroot, nb);
436         saveBookToFile(nb);
437         cout << "Book added and saved successfully\n";
438         break;
439     }
440     case 2:
441         cout << "\nEnter Title to search : ";
442         getline(cin >> ws, tit);
443         Search_Hash_Title(tit);
444         break;
445     case 3:
446         cout << "\nEnter Title to search : ";
447         getline(cin >> ws, tit);
448         tit = asLower(tit);
449         Search_BST_Title(troot, tit);
450         break;
451     case 4:
452         cout << "\nEnter Author to search : ";
453         getline(cin >> ws, aut);
454         Search_Hash_Author(aut);
455         break;
456     case 5:
457         cout << "\nEnter Author to search : ";
458         getline(cin >> ws, aut);
459         aut = asLower(aut);
460         Search_BST_Author(aroot, aut);
461         break;
462     case 6:
463         cout << "\nEnter Title to search : ";
464         getline(cin >> ws, tit);
465         cout << "Enter Author to search : ";
466         getline(cin >> ws, aut);
467         Search_Hash_Both(tit, aut);
468         break;
469     case 7:
470         cout << "\nEnter Title to search : ";
471         getline(cin >> ws, tit);
472         cout << "Enter Author to search : ";
473         getline(cin >> ws, aut);
474         tit = asLower(tit), aut = asLower(aut);

```

```

475         Search_BST_Both(troot, tit, aut);
476         break;
477     case 8:
478         int t;
479         cout << "\nEnter 1 for Title Analysis, 2 for Author
            Analysis : ";
480         cin >> t;
481         if (t == 1 || t == 2) Analyze_Collisions(t);
482         else cout << "Invalid Input!\n";
483         break;
484     case 9:
485         cout << "\nEnter Title to borrow : ";
486         getline(cin >> ws, tit);
487         cout << "Enter Author to borrow : ";
488         getline(cin >> ws, aut);
489         tit = asLower(tit), aut = asLower(aut);
490         Borrow(tit, aut);
491         break;
492     case 10:
493         cout << "\nEnter Title to return : ";
494         getline(cin >> ws, tit);
495         cout << "Enter Author to return : ";
496         getline(cin >> ws, aut);
497         tit = asLower(tit), aut = asLower(aut);
498         Return(tit, aut);
499         break;
500     case 11:
501         cout << "\nExiting... Your data is stored safely.\n";
502         Clean_Memory();
503         break;
504     case 24:
505         importFromCSV(troot, aroot);
506         break;
507     default:
508         cout << "\nInvalid choice! Try again\n";
509         break;
510     }
511 }
512 return 0;
513 }

```


7 Program Outputs

No existing database found. A new one will be created.

```
=====
LIBRARY BOOK SEARCH ENGINE
=====
1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 24
Importing perfectly clean dataset...
Successfully imported 13408 books!
```

```
=====
LIBRARY BOOK SEARCH ENGINE
=====
1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 2
```

```
Enter Title to search : the sun also rises
Searching for 'The Sun Also Rises' in Hash table...
```

```
----- Result # 1 -----
Title   : The Sun Also Rises
Author  : Ernest Hemingway
Year    : 1984
Status  : Available
```

```

-----
----- Result # 2 -----
Title   : The Sun Also Rises
Author   : Ernest Hemingway/william Hurt
Year     : 2006
Status   : Available
-----

Total books found with 'The Sun Also Rises' is : 2

```

```

=====
LIBRARY BOOK SEARCH ENGINE
=====

```

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 3

Enter Title to search : soCIAL anthROPOLogy

```

----- Result # 1 -----
Title   : Social Anthropology
Author   : S.l. Doshi And P.c. Jain
Year     : 2001
Status   : Available
-----

```

```

----- Result # 2 -----
Title   : Social Anthropology
Author   : S.l. Doshi And P.c. Jain
Year     : 2001
Status   : Available
-----

```

```

----- Result # 3 -----
Title   : Social Anthropology
Author   : S.l. Doshi And P.c. Jain
Year     : 2001
Status   : Available

```

```

-----
----- Result # 4 -----
Title   : Social Anthropology
Author  : S.l. Doshi And P.c. Jain
Year    : 2001
Status  : Available
-----

Total books found with 'Social Anthropology' is : 4

```

```

=====
LIBRARY BOOK SEARCH ENGINE
=====

```

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 4

```

Enter Author to search : ram aHUjA
Searching for 'Ram Ahuja' books in Hash table...

```

```

----- Result # 1 -----
Title   : Violence Against Women
Author  : Ram Ahuja
Year    : 1998
Status  : Available
-----

```

```

----- Result # 2 -----
Title   : Indian Social System
Author  : Ram Ahuja
Year    : 1993
Status  : Available
-----

```

```

----- Result # 3 -----
Title   : Indian Social System
Author  : Ram Ahuja
Year    : 1993

```

Status : Available

----- Result # 4 -----
Title : Violence Against Women
Author : Ram Ahuja
Year : 1998
Status : Available

----- Result # 5 -----
Title : Research Methods
Author : Ram Ahuja
Year : 2001
Status : Available

----- Result # 6 -----
Title : Research Methods
Author : Ram Ahuja
Year : 2001
Status : Available

----- Result # 7 -----
Title : Criminology
Author : Ram Ahuja
Year : 2015
Status : Available

----- Result # 8 -----
Title : Criminology
Author : Ram Ahuja
Year : 2015
Status : Available

----- Result # 9 -----
Title : Social Problems In India (fourth Edition)
Author : Ram Ahuja
Year : 2021
Status : Available

----- Result # 10 -----
Title : Social Problems In India (fourth Edition)
Author : Ram Ahuja

Year : 2021
Status : Available

----- Result # 11 -----

Title : Research Methods
Author : Ram Ahuja
Year : 2001
Status : Available

----- Result # 12 -----

Title : Research Methods
Author : Ram Ahuja
Year : 2001
Status : Available

----- Result # 13 -----

Title : Criminology
Author : Ram Ahuja
Year : 2015
Status : Available

----- Result # 14 -----

Title : Criminology
Author : Ram Ahuja
Year : 2015
Status : Available

----- Result # 15 -----

Title : Social Problems In India (fourth Edition)
Author : Ram Ahuja
Year : 2021
Status : Available

Total books found with 'Ram Ahuja' is : 15

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)

```

6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 5

```

Enter Author to search : jAY rUBIN

```

----- Result # 1 -----
Title  : Haruki Murakami And The Music Of Words
Author : Jay Rubin
Year   : 2005
Status : Available
-----

```

```

----- Result # 2 -----
Title  : Haruki Murakami And The Music Of Words
Author : Jay Rubin
Year   : 2005
Status : Available
-----

```

Total books found with 'Jay Rubin' is 2

```

=====
LIBRARY BOOK SEARCH ENGINE
=====

```

```

1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 6

```

Enter Title to search : the other woman
Enter Author to search : jane green
Searching in Hash table for ' The Other Woman' and 'Jane Green'

```

----- Result # 1 -----

```

Title : The Other Woman
Author : Jane Green
Year : 2006
Status : Available

Total books found with 'The Other Woman' wrritten by 'Jane Green' is : 1

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 7

Enter Title to search : DECEption point
Enter Author to search : dan bROWN

----- Result # 1 -----
Title : Deception Point
Author : Dan Brown
Year : 2006
Status : Available

----- Result # 2 -----
Title : Deception Point
Author : Dan Brown
Year : 2005
Status : Available

----- Result # 3 -----
Title : Deception Point
Author : Dan Brown
Year : 2004
Status : Available

Total books found with 'Deception Point' wrritten by 'Dan Brown' is : 3

```
=====
LIBRARY BOOK SEARCH ENGINE
=====
```

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 8

Enter 1 for Title Analysis, 2 for Author Analysis : 1

```
=== Title Hash Table Collision Analysis ===
Total Table Size      : 13681
Empty Slots           : 6182
Slots with 1 Book     : 4270
Slots with collisions : 3229
Longest Chain Length  : 24
Load Factor            : 0.980118
=====
```

```
=====
LIBRARY BOOK SEARCH ENGINE
=====
```

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 8

Enter 1 for Title Analysis, 2 for Author Analysis : 2

=== Author Hash Table Collision Analysis ===

Total Table Size : 13681
Empty Slots : 8008
Slots with 1 Book : 3023
Slots with collisions : 2650
Longest Chain Length : 42
Load Factor : 0.980118

=====

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 9

Enter Title to borrow : amsterdam

Enter Author to borrow : ian mcewan

You borrowed 'Amsterdam' wrritten by 'Ian Mcewan'

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 9

Enter Title to borrow : amsterdam

Enter Author to borrow : ian mcewan
'Amsterdam' wrritten by 'Ian Mcewan' is currently borrowed by someone else
Your book isn't found in our Library

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 10

Enter Title to return : amsterdam
Enter Author to return : ian mann
This book is not belong to our library!

=====

LIBRARY BOOK SEARCH ENGINE

=====

1. Add new Book
 2. Search bt title (HT)
 3. Search by title (BST)
 4. Search by author (HT)
 5. Search by author (BST)
 6. Search by title and author (HT)
 7. Search by title and author (BST)
 8. Analyze hash Collisions
 9. Borrow Book
 10. Return Book
 11. Exit
 24. Import new csv file books
- Enter your choice (1-11) : 10

Enter Title to return : amsterDAM
Enter Author to return : iAN mCEWAN
You returned 'Amsterdam' wrritten by 'Ian Mcewan'

=====

LIBRARY BOOK SEARCH ENGINE

=====

```
1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 10
```

```
Enter Title to return : amsterdam
Enter Author to return : ian mcewan
'Amsterdam' wrritten by 'Ian Mcewan' is already in the library
```

=====

LIBRARY BOOK SEARCH ENGINE

=====

```
1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 1
```

```
Enter Title : Maths
Enter Author : R G Bartle
Enter Year : 2000
Book added and saved successfully
```

=====

LIBRARY BOOK SEARCH ENGINE

=====

```
1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
```

```

5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 7

```

```

Enter Title to search : mATHS
Enter Author to search : r g bartle

```

```

----- Result # 1 -----
Title   : Maths
Author  : R G Bartle
Year    : 2000
Status  : Available
-----

```

```

Total books found with 'Maths' wrritten by 'R G Bartle' is : 1

```

```

=====
LIBRARY BOOK SEARCH ENGINE
=====

```

```

1. Add new Book
2. Search bt title (HT)
3. Search by title (BST)
4. Search by author (HT)
5. Search by author (BST)
6. Search by title and author (HT)
7. Search by title and author (BST)
8. Analyze hash Collisions
9. Borrow Book
10. Return Book
11. Exit
24. Import new csv file books
Enter your choice (1-11) : 11

```

```

Exiting... Your data is stored safely.

```

References

- [1] Wikipedia. *Hash table*. (2026). Available: https://en.wikipedia.org/wiki/Hash_table
- [2] Apxml. *Efficient Data Management at Scale* (2023). Available: <https://apxml.com/courses/how-to-build-a-large-language-model/>

chapter-8-building-managing-large-scale-datasets/
data-indexing-efficient-retrieval

- [3] GeeksforGeeks. *Binary Search Tree Data Structure*. (2026). Available: <https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/>
- [4] CP-Algorithms. *String Hashing (Polynomial Rolling Hash)*. (2024). Available: <https://cp-algorithms.com/string/string-hashing.html>